Hackathon IA pour tous : Vidéo → Résumé & Cours

Équipe de 3 personnes, durée : 3 heures, livrable : un prototype fonctionnel

1. Énoncé du projet

Thème : l'IA pour tous.

Problème : les étudiants, autodidactes et professionnels de la tech en Afrique ont accès à beaucoup de vidéos de formation, mais manquent de temps et de data pour les regarder en entier. Les contenus sont souvent en anglais et difficiles à réviser.

Solution : une application web où l'utilisateur dépose une vidéo (ou un fichier audio). L'application la transcrit, puis génère automatiquement :

- un **résumé** (synthèse, points clés, termes importants) ;
- un **cours structuré** (objectifs, modules, exercices, quiz avec corrigé).

Les résultats sont téléchargeables en texte léger (Markdown) et consultables hors ligne, dans la langue et au niveau choisis par l'utilisateur.

Public cible : toute personne qui apprend ou travaille à partir de vidéos, y compris les développeurs.

MVP (obligatoire) : upload → transcription → résumé → cours → téléchargement.

Hors périmètre : comptes utilisateurs, base de données, vidéos de plus de 10 minutes, langues africaines (en feuille de route).

2. Stack technique

Couche	Choix
Langage	Python 3.10+
Interface	Streamlit
Vidéo → audio	ffmpeg
Audio → texte	Groq API (whisper-large-v3)

Couche	Choix
Résumé et cours	LLM via API (Llama 3.3 70B sur Groq)
Versionnage	Git + GitHub
Déploiement	Streamlit Community Cloud

Architecture : Vidéo → ffmpeg → Audio → Whisper → Transcription → Résumé et Cours

Structure du dépôt : `app.py`, `pipeline.py`, `prompts.py`, `requirements.txt`, `packages.txt`, `.gitignore` (avec `.env`).

3. Contrat d'interface (à valider dans les 15 premières minutes)

- `extraire_audio(video_path) -> audio_path` (Personne A)
- `transcrire(audio_path) -> texte` (Personne A)
- `generer(texte, consigne, langue, niveau) -> str` (Personne B)

La Personne C développe l'interface avec des fausses données, puis branche les vraies fonctions.

4. Répartition des tâches

Personne	Rôle	Responsabilités	Fichiers
A	Pipeline	Extraction audio, transcription, gestion des erreurs et des gros fichiers	`pipeline.py`
B	IA et contenu	Prompts du résumé et du cours, qualité des sorties, langues et niveaux, partie « problème et impact » du pitch	`prompts.py`, `generer()`
C	Interface, déploiement, démo	Interface Streamlit, GitHub, déploiement, slides et démo en direct	`app.py`, `requirements.txt`, `packages.txt`

5. Plan de travail

Horaire	Phase	A	B	C
0h00 à 0h30	Cadrage	Clé API Groq, premier test ffmpeg	Clé API, premier appel LLM	Dépôt GitHub, structure du projet, `.gitignore`
0h30 à 1h30	Développement parallèle	Audio + transcription, test sur 3 vidéos	Prompts résumé et cours, `generer()`	Interface complète avec fausses données
1h30	Synchro (10 min)	Chacun montre son livrable		
1h30 à 2h30	Intégration	Erreurs, découpage des fichiers longs si possible	Amélioration des prompts, tests FR et EN	Branchement des vraies fonctions, déploiement
2h30	Gel du code	Test complet de bout en bout, tous ensemble		
2h30 à 3h00	Finition et démo	Vidéo d'exemple et vidéo de secours	Texte du pitch	Slides (3 max) et démo

6. Plan de présentation (2 à 3 minutes)

Qui	Contenu	Durée
B	Le problème : accès aux vidéos, temps, data	30 s
C	Démo en direct : upload, résumé, cours	1 min 30
A	Architecture et limites honnêtes	30 s
Tous	Feuille de route : langues africaines, mode hors ligne, vidéos longues	20 s

7. Règles d'équipe

1. Une branche Git par personne, fusion dans `main` aux points de synchro.
2. Ne pas modifier le fichier d'un autre sans le prévenir.
3. Ne jamais mettre la clé API dans le code ni sur GitHub.
4. Si quelqu'un est bloqué plus de 15 minutes, il le dit immédiatement.
5. Utiliser une vidéo de test commune (3 minutes, en français).

8. Priorités en cas de manque de temps

À sacrifier dans cet ordre : (1) le découpage des longues vidéos, (2) l'onglet Transcription, (3) le déploiement web (rester en local avec la vidéo de secours).

À ne jamais sacrifier : upload → résumé → cours.

9. Feuille de route (après le hackathon)

- Support des langues africaines (swahili, wolof, etc.).
- Mode hors ligne avec modèles locaux, pour les écoles et centres à faible connectivité.
- Traitement des vidéos longues par découpage.
- Lecture à voix haute et export PDF.